

UNIVERSIDAD DE EL SALVADOR

Facultad Multidisciplinaria De Occidente Departamento
de Ingeniería y Arquitectura Ingeniería en Desarrollo de
Software

Asignatura: Manejo Y Estructura de datos

Tema: Guía 3

Alumna:

Alyson Pamela Alvarado Jovel
(AJ24002)

G
U
I
A

3

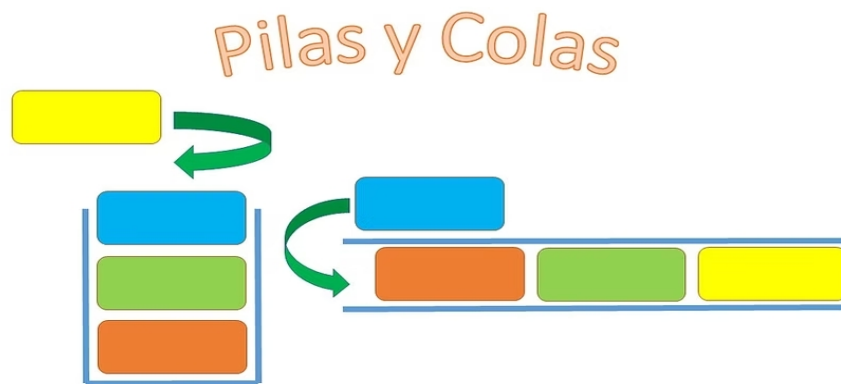


Resumen de Semana 13 y 14

Semana 13 – Pilas y Colas

Durante la semana 13 se estudiaron dos estructuras de datos lineales: pilas (stack) y colas (queue). Las pilas funcionan bajo el principio LIFO (Last In, First Out), permitiendo el acceso solo por el tope. Las operaciones principales son push (insertar), pop (eliminar) y peek o first (consultar sin eliminar). Se utilizan en el cálculo de expresiones, funciones recursivas, y más.

Por otro lado, las colas siguen el principio FIFO (First In, First Out), insertando elementos por el final y eliminándolos por el frente. Se implementan mediante vectores o listas enlazadas, y son esenciales en sistemas como la gestión de tareas, impresión y comunicaciones entre dispositivos.



Semana 14 – Recursividad

La semana 14 estuvo dedicada a la recursividad, una técnica en la cual una función se llama a sí misma. Se compone de un caso base y una llamada recursiva a un subproblema más simple.

Ventajas: permite soluciones más naturales, claras y elegantes para problemas complejos.
Desventajas: uso elevado de memoria y riesgo de desbordamiento de pila.

Las funciones recursivas pueden ser directas (se llaman a sí mismas) o indirectas (se llaman entre sí). Comparada con la iteración, la recursividad es menos eficiente pero más legible en ciertos casos. Se recomienda cuando el rendimiento no es una prioridad y se valora la claridad del código.

Ejemplo de Pila en Java

```
import java.util.Stack;

public class EjemploPila {
    public static void main(String[] args) {
        Stack<String> pila = new Stack<>();

        pila.push("Libro 1");
        pila.push("Libro 2");
        pila.push("Libro 3");

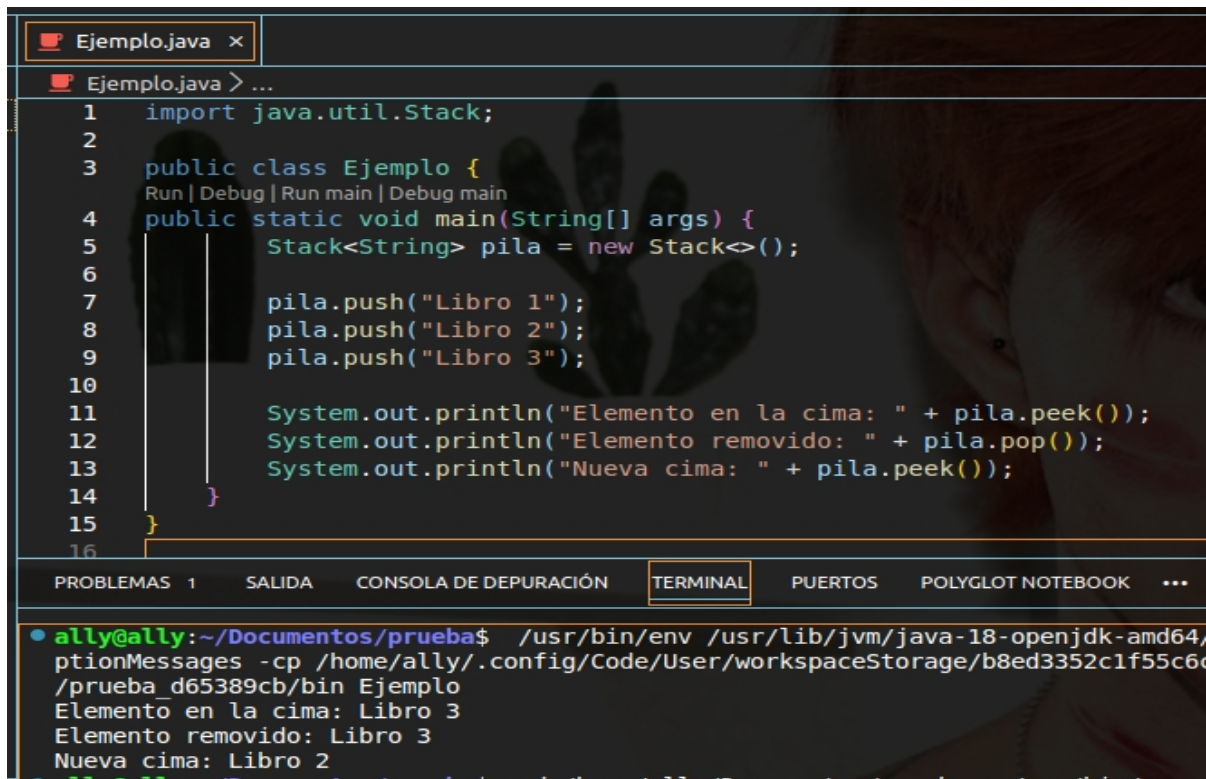
        System.out.println("Elemento en la cima: " + pila.peek());
        System.out.println("Elemento removido: " + pila.pop());
        System.out.println("Nueva cima: " + pila.peek());
    }
}
```

Salida esperada:

Elemento en la cima: Libro 3

Elemento removido: Libro 3

Nueva cima: Libro 2



The screenshot shows an IDE window titled 'Ejemplo.java'. The code is as follows:

```
1 import java.util.Stack;
2
3 public class Ejemplo {
4     public static void main(String[] args) {
5         Stack<String> pila = new Stack<>();
6
7         pila.push("Libro 1");
8         pila.push("Libro 2");
9         pila.push("Libro 3");
10
11         System.out.println("Elemento en la cima: " + pila.peek());
12         System.out.println("Elemento removido: " + pila.pop());
13         System.out.println("Nueva cima: " + pila.peek());
14     }
15 }
16
```

Below the code editor, the 'TERMINAL' tab is active, showing the command to run the program and its output:

```
ally@ally:~/Documentos/prueba$ /usr/bin/env /usr/lib/jvm/java-18-openjdk-amd64/bin/java -cp /home/ally/.config/Code/User/workspaceStorage/b8ed3352c1f55c6d/prueba_d65389cb/bin Ejemplo
Elemento en la cima: Libro 3
Elemento removido: Libro 3
Nueva cima: Libro 2
```

Ejemplo de Cola en Java

```
import java.util.LinkedList;
import java.util.Queue;

public class EjemploCola {
    public static void main(String[] args) {
        Queue<String> cola = new LinkedList<>();

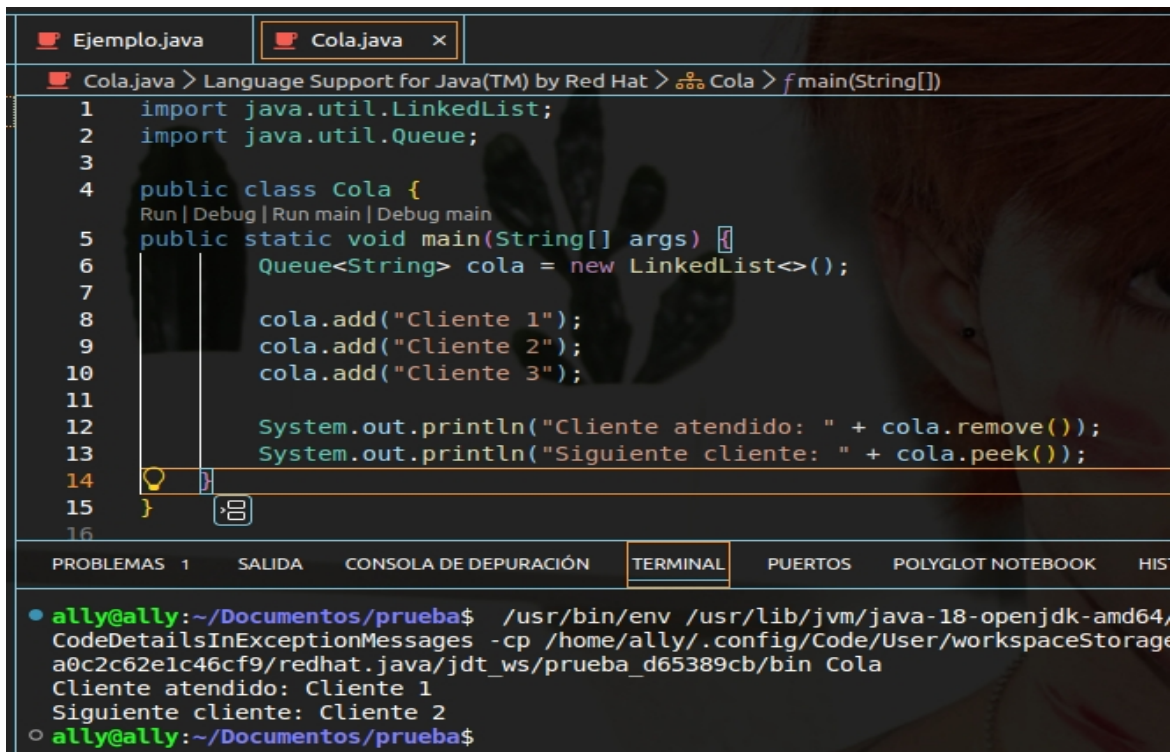
        cola.add("Cliente 1");
        cola.add("Cliente 2");
        cola.add("Cliente 3");

        System.out.println("Cliente atendido: " + cola.remove());
        System.out.println("Siguiente cliente: " + cola.peek());
    }
}
```

Salida esperada:

Cliente atendido: Cliente 1

Siguiente cliente: Cliente 2



The screenshot shows an IDE with two tabs: 'Ejemplo.java' and 'Cola.java'. The 'Cola.java' tab is active, displaying the following code:

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 public class Cola {
5     public static void main(String[] args) {
6         Queue<String> cola = new LinkedList<>();
7
8         cola.add("Cliente 1");
9         cola.add("Cliente 2");
10        cola.add("Cliente 3");
11
12        System.out.println("Cliente atendido: " + cola.remove());
13        System.out.println("Siguiente cliente: " + cola.peek());
14    }
15 }
```

Below the code editor, the 'TERMINAL' tab is active, showing the execution output:

```
ally@ally:~/Documentos/prueba$ /usr/bin/env /usr/lib/jvm/java-18-openjdk-amd64/
CodeDetailsInExceptionMessages -cp /home/ally/.config/Code/User/workspaceStorage
a0c2c62e1c46cf9/redhat.java/jdt_ws/prueba_d65389cb/bin Cola
Cliente atendido: Cliente 1
Siguiente cliente: Cliente 2
ally@ally:~/Documentos/prueba$
```

Bibliografía

- Rodríguez L. (2012). Fundamentos de Programación II. Universidad Pontificia de Salamanca.
- Mary E. (2015). Estructura de datos.
- Deitel, H. M., & Deitel, P. J. (2015). Java: How to Program. Pearson Education.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms. MIT Press.
- Oracle Java Documentation: <https://docs.oracle.com/javas>